
doc_id: api-reference

title: API Reference

version: 1.0

module: api

last_updated: 2026-09-12

API Terminal (#api-terminal)

All API requests must use HTTPS.

```
https://api.askmydocs.example/v1
```

The API is versioned in the URL path. The current version is v1. Breaking changes are announced at least 6 months in advance.

Authentication (#authentication)

Every request must include an Authorization header with a bearer token:

```
GET /v1/projects HTTP/1.1
Host: api.askmydocs.example
Authorization: Bearer eyJhbGciOiJSUzI1NiIs...
```

Tokens expire after 60 minutes. Use the refresh flow before expiry to avoid 401 responses.

Pagination (#pagination)

List endpoints paginate using cursor-based pagination. The response contains a next_cursor field; pass it as the cursor parameter to fetch the next page.

Idempotency (#idempotency)

Write endpoints accept an Idempotency-Key header. If a request with the same key is retried, the server returns the original response without applying the change twice.

Idempotency keys are honored for 24 hours. Use UUID v4 values.

Rate Limits (#rate-limits)

The default rate limit is 100 requests per minute. Short bursts up to 300 requests per minute are allowed.

Rate limits apply per API key, per endpoint. Responses include X-RateLimit-Limit, X-RateLimit-Remaining, and X-RateLimit-Reset headers.

Warning: Hitting the rate limit returns HTTP 429 with a Retry-After header. Respect the header; do not retry immediately.

Webhooks (#webhooks)

Webhooks notify your system about events such as document processed, document failed, and ingestion complete.

Each delivery is signed with HMAC-SHA256 signature using the webhook secret. The signature is sent in the X-AskMyDocs-Signature header as t=,v1=.

Delivery is retried up to 5 retries, exponential backoff, max 10 minutes.

```
# Verify a webhook signature
import hashlib, hmac
def verify(secret, body, header):
    timestamp, signature = header.split(',')
    digest = hmac.new(secret.encode(), f'{timestamp}.{body}'.encode(), hashlib.sha
256).hexdigest()
    return hmac.compare_digest(digest, signature.split('=')[1])
```

Common Headers (#common-headers)